LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT)



Disusun oleh :

Nama : Restu anggara

NPM : 25782051

Kelas : TRI-3B

Dosen:

Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.

Agus Ambarwari, S.Pd.,M.Kom.

Andi Saptono,S.Kom

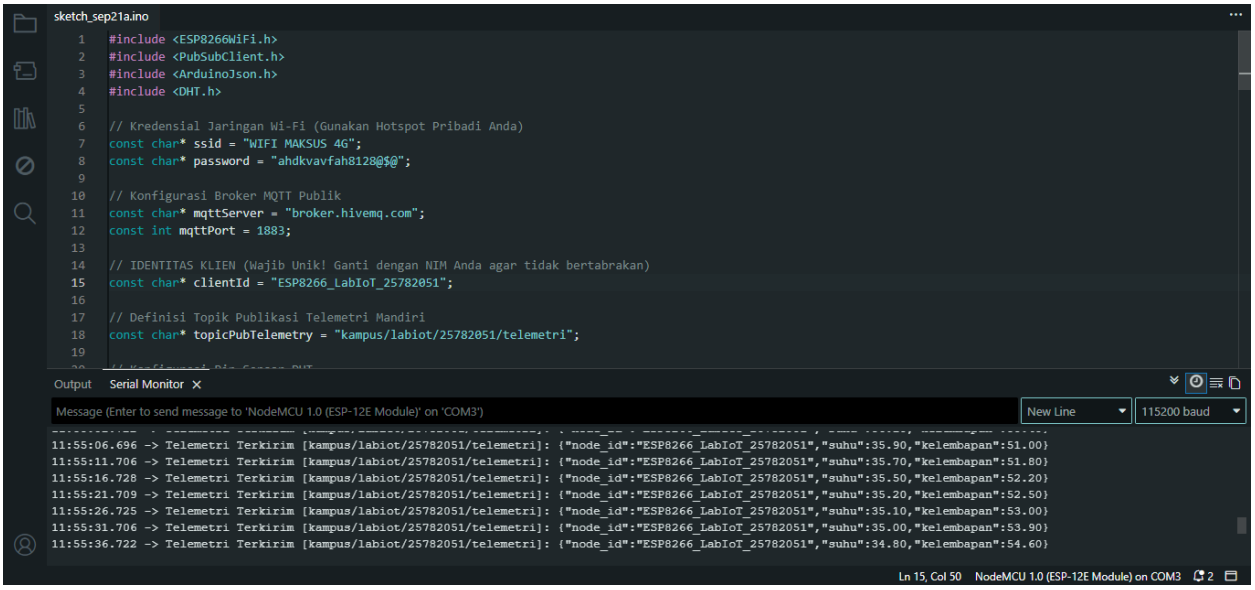
TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON




```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "WIFI MAKSUS 4G";
const char* password = "ahdkvavfah8128@$@";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782051";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782051/telemetri";

// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;    // Pin D4 (GPIO 2)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
```

```
unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000; // Interval pengiriman 5 detik

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
    } else {
      Serial.print(" Gagal, kode status rc=");
      Serial.print(client.state());
      Serial.println(". Mencoba kembali dalam 5 detik...");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
}

void loop() {
  if (!client.connected()) {
    reconnectMQTT();
```

```
  }
  client.loop(); // Memelihara sesi TCP dan keep-alive ping

  unsigned long currentMillis = millis();
  if (currentMillis - previousPublishTime >= publishInterval) {
    previousPublishTime = currentMillis;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    // Proteksi pembacaan sensor DHT
    if (isnan(t) || isnan(h)) {
      Serial.println("Peringatan: Gagal membaca data DHT!");
      return;
    }

    // Alokasi dokumen JSON statis
    StaticJsonDocument<256> doc;
    doc["node_id"] = clientId;
    doc["suhu"] = serialized(String(t, 2));       // Format desimal 2 angka di
belakang koma
    doc["kelembapan"] = serialized(String(h, 2));

    // Serialisasi JSON ke dalam array buffer karakter
    char jsonBuffer[256];
    serializeJson(doc, jsonBuffer);

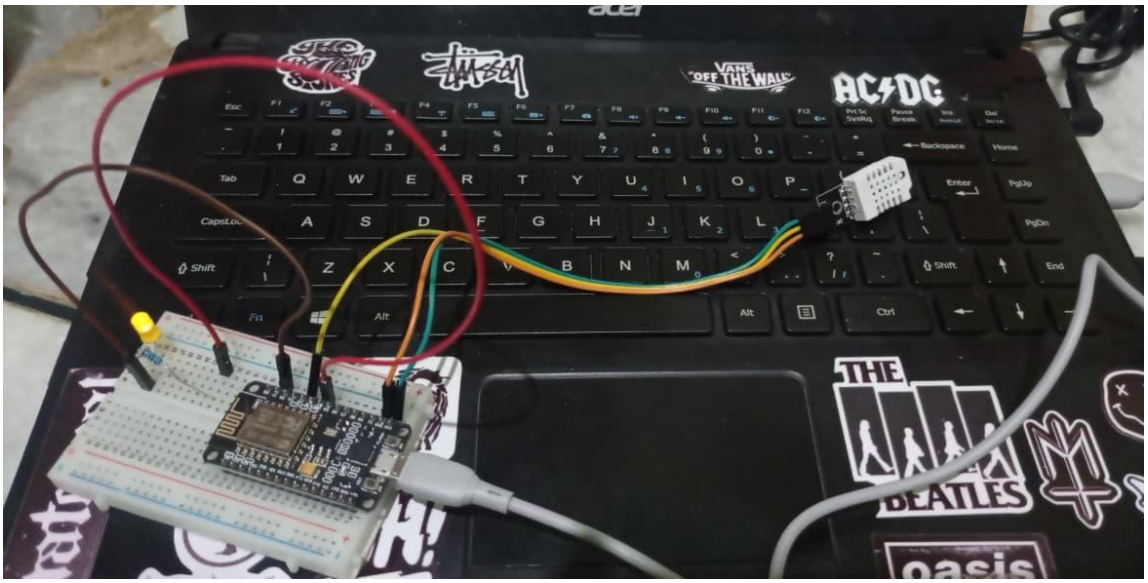
    // Publikasi paket data ke Broker
    bool isSuccess = client.publish(topicPubTelemetry, jsonBuffer);
    if (isSuccess) {
      Serial.print("Telemetri Terkirim [");
      Serial.print(topicPubTelemetry);
      Serial.print("]: ");
      Serial.println(jsonBuffer);
    } else {
      Serial.println("Peringatan: Gagal memublikasikan paket!");
    }
  }
}
```

Penjelasan:

Kegunaan utama dari kode program di atas adalah untuk membangun sebuah perangkat *Internet of Things* (IoT) berbasis mikrokontroler ESP8266 yang mampu membaca kondisi lingkungan secara otomatis dan mengirimkannya ke internet. Melalui sensor DHT22 yang terpasang,

perangkat ini mengukur data suhu dan kelembapan secara berkala setiap 5 detik, lalu mengemas data mentah tersebut ke dalam format standar **JSON** agar mudah dibaca oleh sistem lain. Selanjutnya, menggunakan protokol komunikasi ringan **MQTT**, data berformat JSON tersebut dipublikasikan secara *real-time* ke broker publik (broker.hivemq.com) menggunakan topik khusus berbasis NPM (25782051) agar tidak terjadi tabrakan data antarperangkat. Secara keseluruhan, kode ini berfungsi sebagai jembatan yang mengubah data fisik lingkungan (suhu dan kelembapan) menjadi informasi digital terstruktur yang siap diolah atau dipantau dari jarak jauh melalui jaringan cloud.

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron



Praktikum_1 | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

Praktikum_1.ino

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "WIFI MAKSUS 4G";
const char* password = "ahdkvavfah8128@$@";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782051_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "k";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)
```

Output Serial Monitor

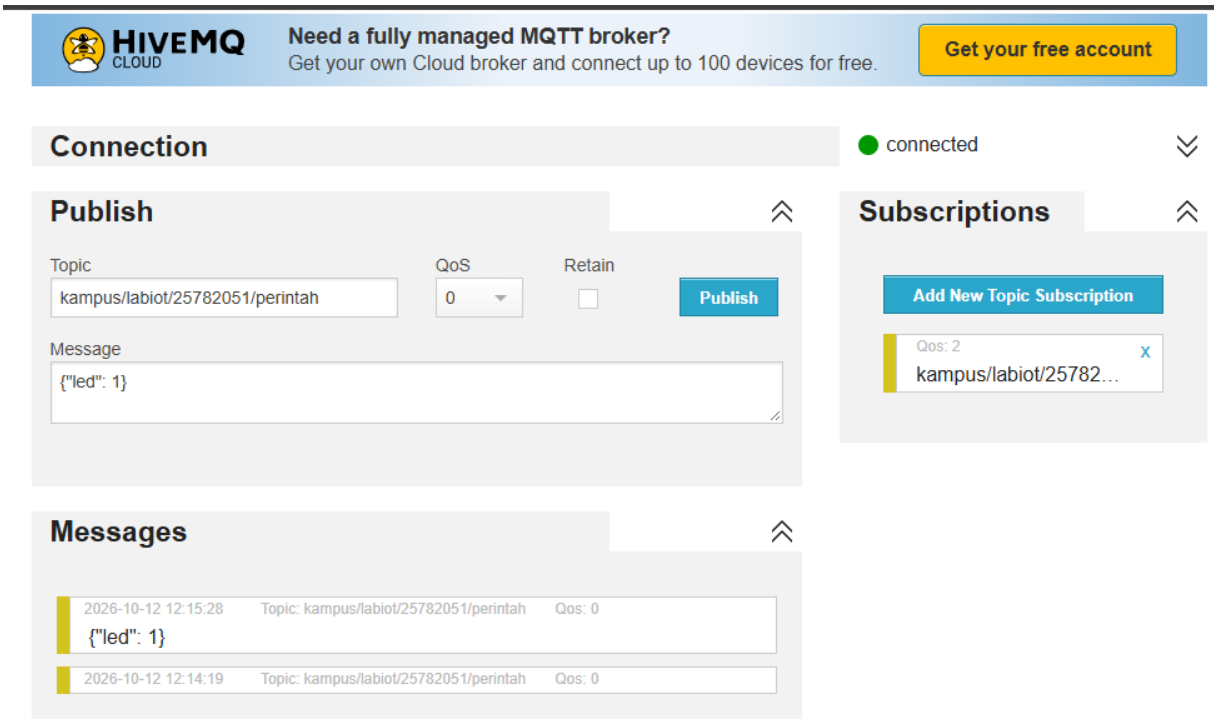
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

```
12:14:19.273 -> Pesan Baru Masuk pada Topik: kampus/labiot/25782051/perintah
12:14:19.407 -> Payload Diterima:
12:14:19.407 -> Gagal parsing JSON: EmptyInput
12:15:28.516 -> ====================================
12:15:28.516 -> Pesan Baru Masuk pada Topik: kampus/labiot/25782051/perintah
12:15:28.516 -> Payload Diterima: {"led": 1}
12:15:28.516 -> --> Aksi: Status LED diubah menjadi MENYALA (ON)
12:15:28.516 -> ====================================
```

Building sketch

Ln 15, Col 33 NodeMCU 1.0 (ESP-12E Module) on COM3



Kode perintah:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "WIFI MAKSUS 4G";
const char* password = "ahdkvavfah8128@$@";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782051_Sub";

// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "k";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);
```

```
void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
}

// Callback otomatis: Berjalan saat mendeteksi paket masuk pada topik terdaftar
void onMessageReceived(char* topic, byte* payload, unsigned int length) {
  Serial.println("=========================================");
  Serial.print("Pesan Baru Masuk pada Topik: ");
  Serial.println(topic);

  // Ubah deretan byte payload menjadi string aman dengan null-terminator
  char payloadStr[length + 1];
  memcpy(payloadStr, payload, length);
  payloadStr[length] = '\0';

  Serial.print("Payload Diterima: ");
  Serial.println(payloadStr);

  // Deserialisasi (Parsing) JSON
  StaticJsonDocument<256> doc;
  DeserializationError error = deserializeJson(doc, payloadStr);

  if (error) {
    Serial.print("Gagal parsing JSON: ");
    Serial.println(error.f_str());
    return;
  }

  // Evaluasi Parameter 'led' (Format yang diharapkan: {"led": 1} atau {"led":
0})
  if (doc.containsKey("led")) {
    int stateLED = doc["led"];
    digitalWrite(ledPin, stateLED == 1 ? HIGH : LOW);
    Serial.printf("--> Aksi: Status LED diubah menjadi %s\n", stateLED == 1 ?
"MENYALA (ON)" : "PADAM (OFF)");
  }
  Serial.println("=========================================");
```

```
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Mencoba koneksi MQTT...");
    if (client.connect(clientId)) {
      Serial.println(" Terhubung!");
      // Segera daftarkan subskripsi topik perintah sesaat setelah koneksi pulih
      client.subscribe(topicSubCommand);
      Serial.print("Subskripsi Topik: ");
      Serial.println(topicSubCommand);
    } else {
      Serial.printf("Gagal (rc=%d). Ulangi dalam 5 detik...\n", client.state());
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  // Status awal LED padam
  digitalWrite(ledPin, LOW);

  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
  client.setCallback(onMessageReceived); // Daftarkan fungsi callback
}

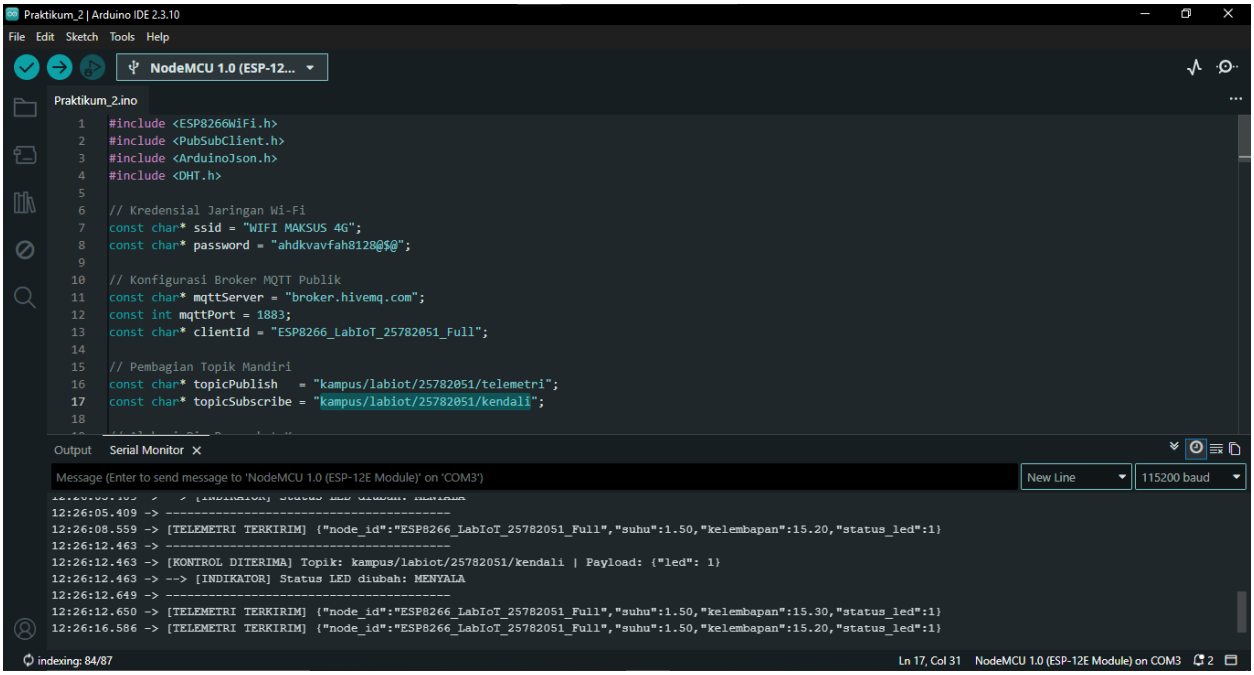
void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }
  client.loop(); // Wajib untuk mendengarkan pesan masuk
}
```

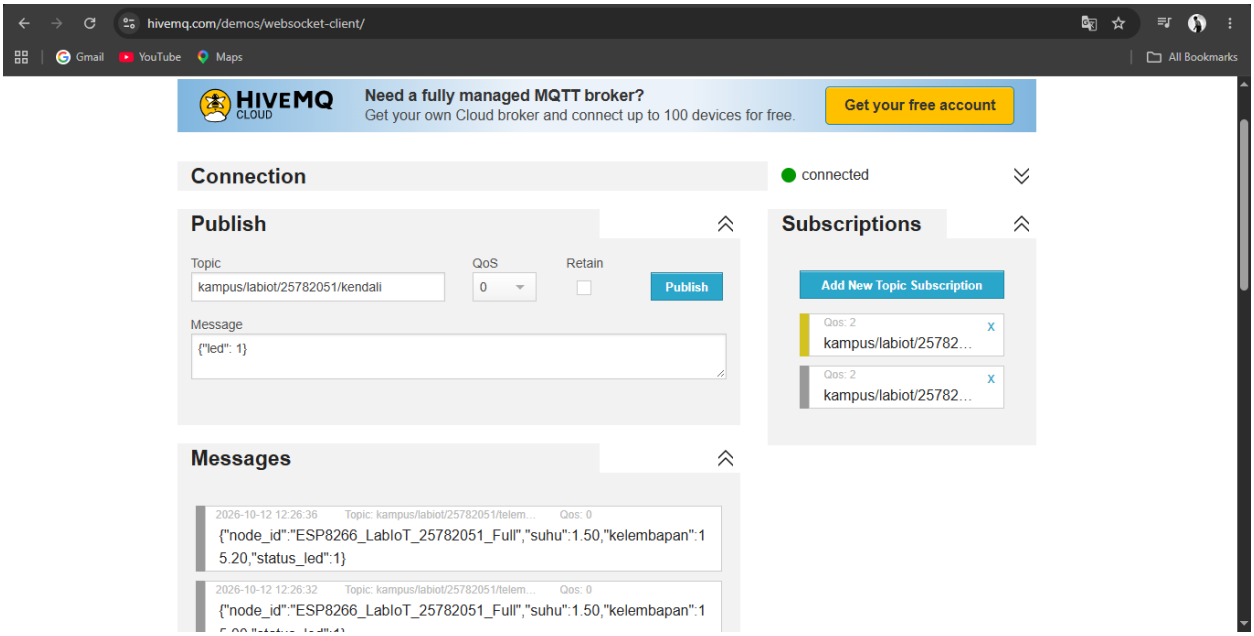
Penjelasan:

lanjutan dari praktikum 1 yang berfokus pada penerimaan perintah melalui subskripsi MQTT (*subscriber*) berformat JSON untuk mengontrol perangkat keras, yaitu LED, menggunakan mikrokontroler ESP8266. Program ini bekerja dengan menghubungkan perangkat ke jaringan Wi-Fi dan broker MQTT publik (broker.hivemq.com), lalu mendengarkan pesan perintah masuk pada topik tertentu. Ketika sebuah pesan berformat JSON seperti {"led": 1} atau {"led": 0} diterima

melalui broker, fungsi *callback* (onMessageReceived) akan secara otomatis memparsing data tersebut. Berdasarkan nilai parameter yang terkandung di dalam JSON, mikrokontroler akan langsung mengeksekusi perubahan status pin fisik untuk menyalakan atau memadamkan LED secara *real-time*. Secara keseluruhan, kode ini berfungsi untuk mewujudkan sistem kendali jarak jauh berbasis *Cloud* di mana perangkat IoT dapat menerima dan menerjemahkan perintah terstruktur untuk mengontrol aktuator fisik.

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)





Kode perintah:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>
```

```
// Kredensial Jaringan Wi-Fi
const char* ssid = "WIFI MAKSUS 4G";
const char* password = "ahdkvavfah8128@$@";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782051_Full";

// Pembagian Topik Mandiri
const char* topicPublish   = "kampus/labiot/25782051/telemetri";
const char* topicSubscribe = "kampus/labiot/25782051/kendali";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus kirim 4 detik

void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic,
incomingBuffer);

  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  if (doc.containsKey("led")) {
    int val = doc["led"];
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
```

```
    Serial.printf("--> [INDIKATOR] Status LED diubah: %s\n", val == 1 ? "MENYALA"
: "PADAM");
  }
  Serial.println("----------------------------------------");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker...");
    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik: %s\n", topicSubscribe);
    } else {
      Serial.printf("Gagal (rc=%d). Coba lagi dalam 4 detik...\n",
client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
```

```
    reconnect();
  }
  client.loop(); // Memelihara sesi TCP dan memproses callback masuk

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);

      char buffer[256];
      serializeJson(docOut, buffer);

      bool success = client.publish(topicPublish, buffer);
      if (success) {
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri!");
      }
    }
  }
}
```

Penjelasan:

Kode ini bekerja dengan menghubungkan perangkat mikrokontroler ke jaringan Wi-Fi dan broker MQTT publik (broker.hivemq.com) menggunakan identitas klien unik ESP8266_LabIoT_25782051_Full. Di dalam program, didefinisikan dua topik terpisah, yaitu topicPublish (kampus/labiot/25782051/telemetri) untuk mengirim data sensor dan topicSubscribe (kampus/labiot/25782051/kendali) untuk mendengarkan perintah masuk. Melalui fungsi loop() yang berjalan secara berkala setiap 4 detik, perangkat membaca suhu dan kelembapan dari sensor DHT, membungkus data tersebut bersama status LED saat ini ke dalam format JSON, lalu mempublikasikannya ke broker. Secara bersamaan, fungsi *callback* (callback) selalu siap siaga menangani pesan perintah masuk (misalnya berformat {"led": 1}) dari platform eksternal seperti HiveMQ WebSocket Client, memparsing isinya, dan langsung mengubah status fisik pin LED (OUTPUT pada pin D1/GPIO 5) menjadi menyala atau padam.

6.6 Latihan

- Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):
  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.

     MQTT lebih hemat bandwidth karena ukuran header pesan lebih kecil dibandingkan HTTP. MQTT cocok untuk perangkat IoT yang menggunakan paket data seluler terbatas.

  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.

     MQTT memiliki format pesan sederhana dan koneksi yang dipertahankan, sehingga pemrosesan komunikasi pada mikrokontroler lebih ringan dibandingkan HTTP.

  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

     MQTT menggunakan sistem publish-subscribe melalui broker. Pengirim cukup mengirim satu pesan ke topic, lalu broker meneruskannya ke semua perangkat yang berlangganan.

- Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
  Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

  i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.

     pabrik/+/+/suhu

     Penjelasan: Tanda + mewakili satu tingkat topik. Jadi, sistem menerima data suhu dari semua mesin di line1, line2, dan line3.

  ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.

     pabrik/line2/m1/+

     Penjelasan: Tanda + mencakup semua sensor pada mesin m1 di line2, yaitu suhu, getaran, dan status.

  iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

pabrik/#

Penjelasan: Tanda # mencakup semua tingkat topik di bawah pabrik, sehingga semua data sensor dari seluruh lini dan mesin dapat diterima.

- Latihan 3 (Implementasi Serialisasi Data Dinamis): Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

  i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

  ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

```
// Alokasi dokumen JSON statis
StaticJsonDocument<256> doc;
doc["node_id"] = clientId;
doc["suhu"] = serialized(String(t, 2));
doc["kelembapan"] = serialized(String(h, 2));

// Modifikasi kode
doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();

// Serialisasi JSON ke dalam array buffer karakter
char jsonBuffer[256];
serializeJson(doc, jsonBuffer);
```

1. **doc["uptime_detik"] = millis() / 1000;** digunakan untuk menghitung lama waktu ESP8266 aktif dalam satuan detik sejak perangkat dinyalakan.

2. **doc["ip_lokal"] = WiFi.localIP().toString();** digunakan untuk mengambil alamat IP lokal ESP8266 dan menyimpannya dalam format string ke dalam JSON.

- Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking): Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

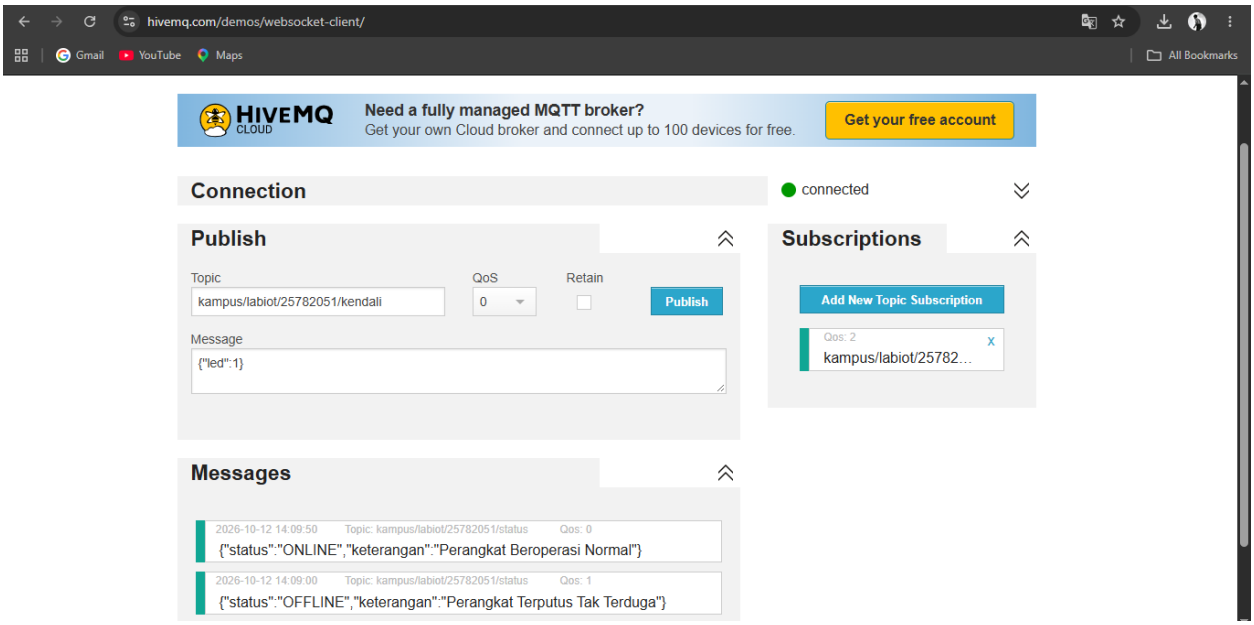
  i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

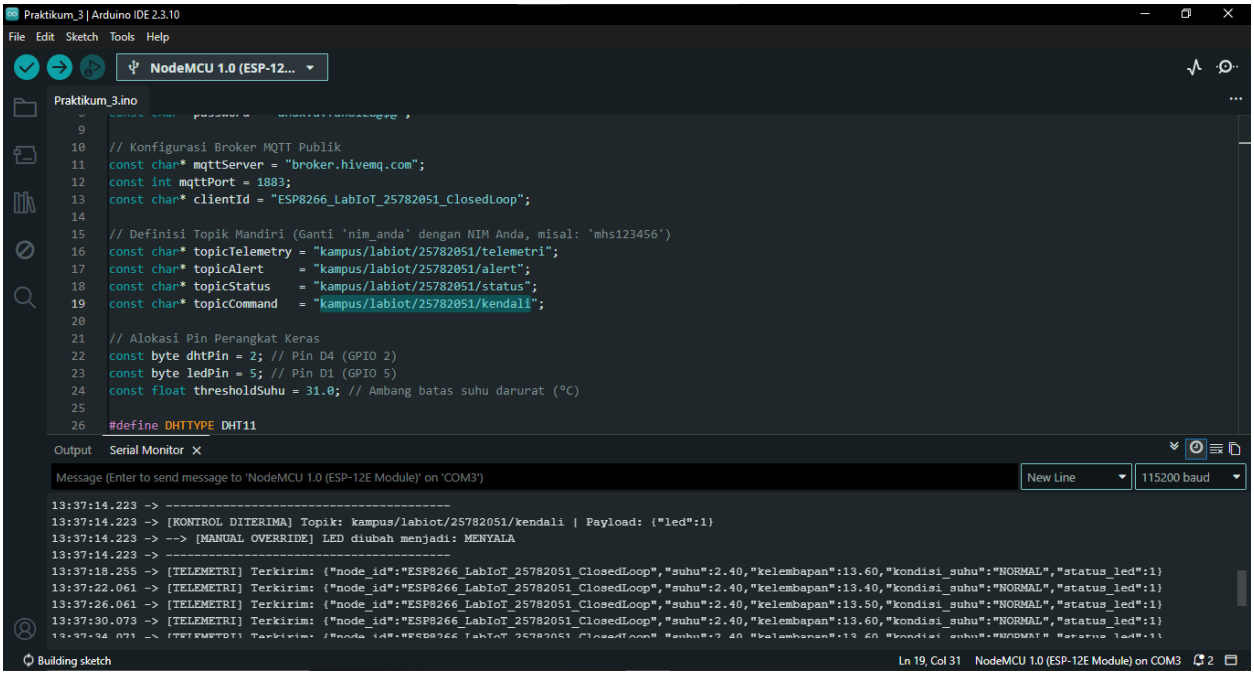
  Jika delay(6000) ditambahkan ke fungsi loop(), ESP8266 akan berhenti menjalankan proses utama selama 6 detik. Akibatnya, client.loop() tidak dipanggil selama waktu tersebut sehingga penerimaan pesan MQTT dan eksekusi callback menjadi terlambat. Perintah untuk mengontrol LED juga tidak langsung diproses.

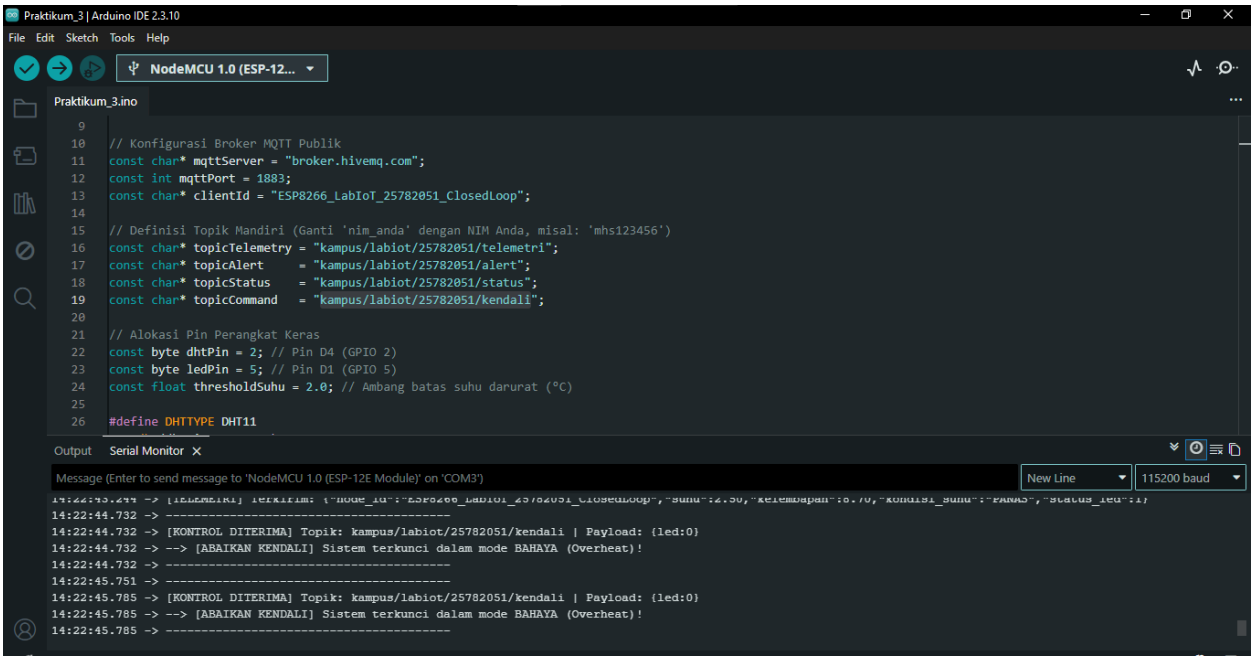
ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Fungsi reconnect() menggunakan perulangan while (!client.connected()) untuk memeriksa koneksi MQTT. Jika koneksi terputus, ESP8266 mencoba terhubung kembali ke broker. Jika berhasil, perangkat berlangganan kembali ke topik kendali menggunakan client.subscribe(topicSubscribe). Jika gagal, perangkat menunggu 4 detik sebelum mencoba lagi.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri







Penjelasan:

pengembangan tingkat lanjut dari Praktikum 3 yang bertujuan untuk mengubah sistem IoT menjadi sarana pengawasan keselamatan industri yang cerdas, otonom, dan tahan banting. Pengembangan utama dari praktikum sebelumnya mencakup penambahan klasifikasi status lingkungan secara dinamis pada data telemetri rutin ("NORMAL" atau "PANAS" berdasarkan ambang batas suhu 31.0°C), serta penerapan sistem kecerdasan lokal (*edge safety engine*) yang membuat mikrokontroler dapat secara otonom mempublikasikan peringatan bahaya ke topik khusus (alert) dan menyalakan LED darurat tanpa menunggu instruksi operator ketika terjadi *overheat*. Selain itu, sistem ini dilengkapi dengan mekanisme ketahanan jaringan *Last Will and Testament* (LWT) agar broker MQTT dapat secara otomatis menyiarkan status OFFLINE jika perangkat terputus secara mendadak, sementara fleksibilitas kendali jarak jauh manual (*override*) tetap dipertahankan bagi operator melalui topik kendali selama kondisi suhu berada di batas aman.